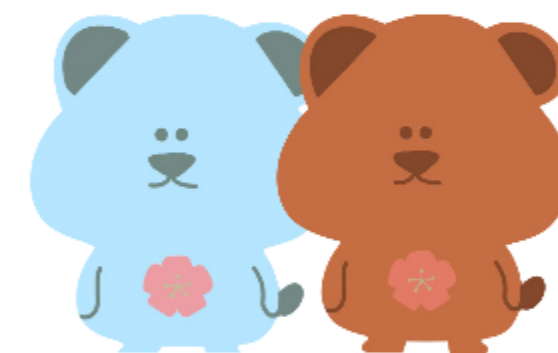




EURON 9주차

3팀 강유진 김재희 박혜린

Vit



#01 Vit

#1 Vit

- 트랜스포머 구조 자체를 컴퓨터 비전 분야에 적용한 첫번째 연구
- 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력됨
- 시퀀셜 배열을 가정함 (왼→오른, 위→아래)

#2 BERT

문장보다 작은 단위의 토큰들이 순차적으로 입력

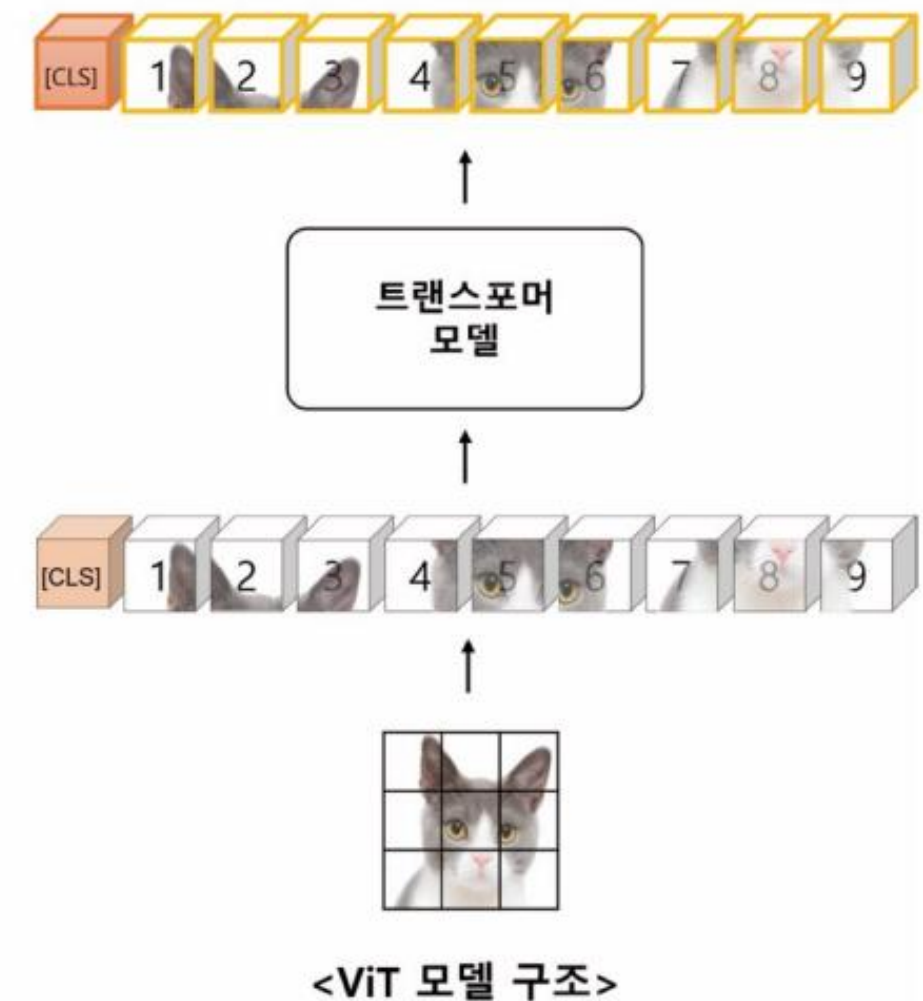
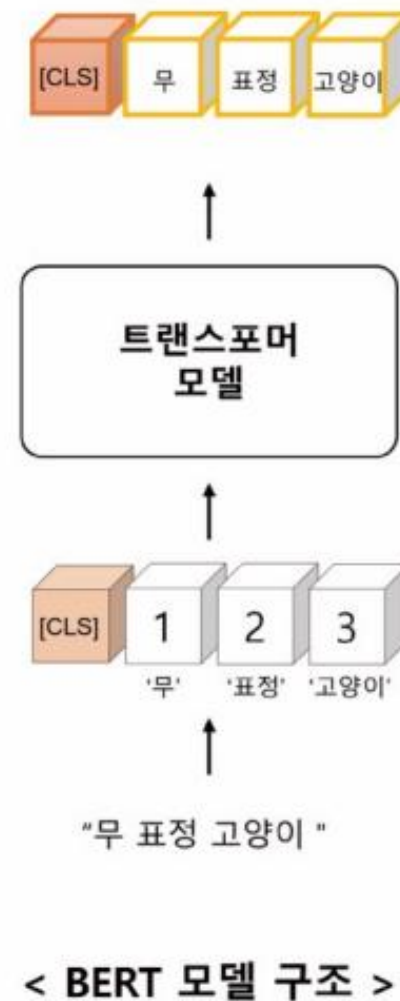
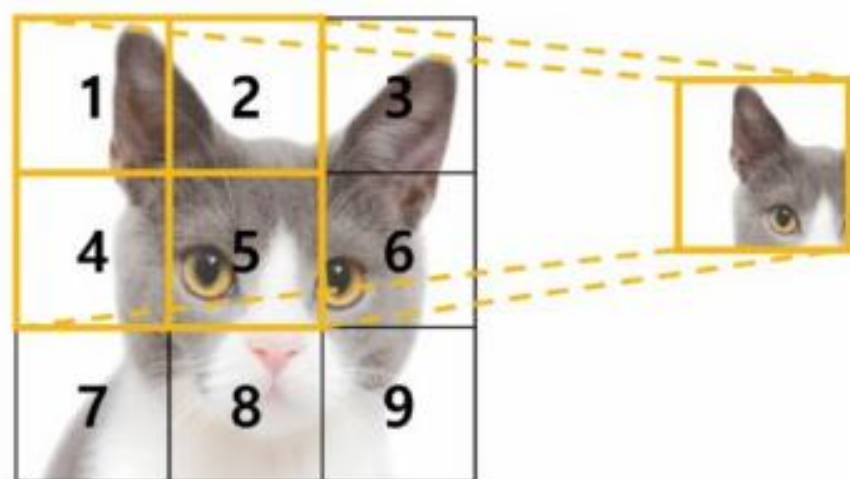


그림 10.2 BERT의 ViT 모델 구조 차이

#02 합성곱 모델과 ViT 모델 비교

합성곱 신경망

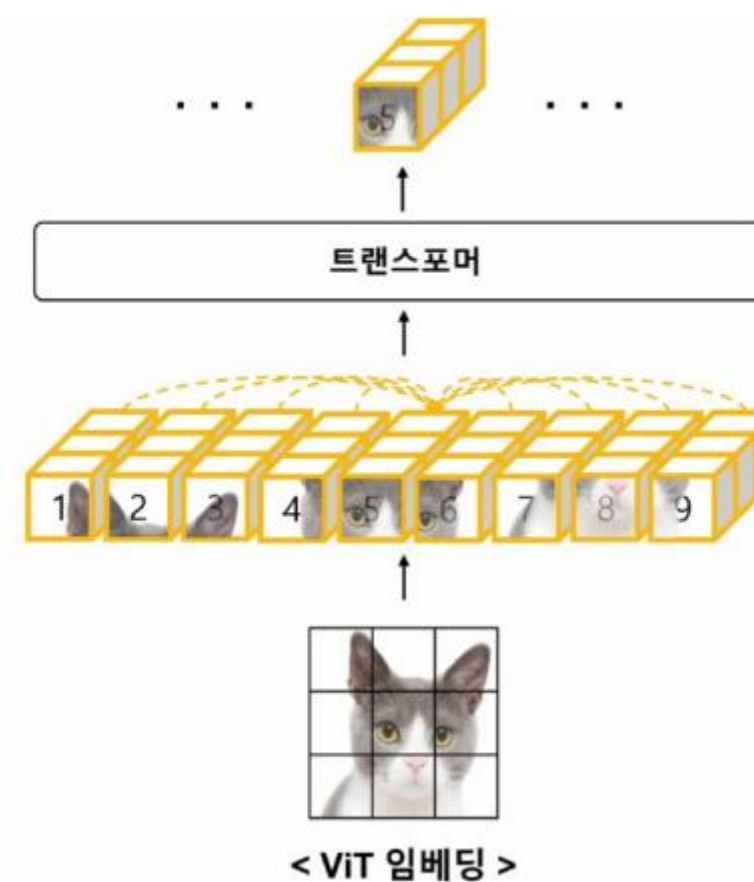
- 이미지 패치 중 일부만 선택해서 학습해서 이미지 전체의 특징 추출
- 목표: 왼쪽 눈의 특징 추출, 해당 부분만 선택해 학습
- 이미지를 픽셀 단위로 처리
- 입력 이미지의 크기 고정
⇒ 크기가 다른 이미지를 처리하려면 **padding** 필요



< CNN 임베딩 >

ViT 임베딩

- 이미지를 작은 패치로 나눠 각 패치 간의 상관관계 학습
- 셀프 어텐션 방법 사용
- 모든 이미지 패치가 학습에 관여
- 어텐션 거리 계산해서 오직 한 개의 ViT 레이어로 전체 이미지 정보를 쉽게 표현 가능
- 패치 단위; 더 작은 모델로도 높은 성능 얻을 수 있음
- 패치 간의 **self-attention** 변환해 취익



이미지

#03 ViT의 귀납적 편향

귀납적 편향 (inductive Bias)

- 일반화 성능 향상을 위한 모델의 가정(assumption)
- 이미지 데이터는 공간적 관계를 잘 표현해서 local 편향을 가진 합성곱 신경망 많이 사용
- 너무 강한 귀납적 편향은 다른 유형의 데이터나 관계를 표현하는데 어려움을 초래할 수 있음

표 10.1 딥러닝 모델의 귀납적 편향

딥러닝 모델	귀납적 편향
합성곱 신경망(CNN)	지역적 편향
순환 신경망(RNN)	순차적 편향
ViT	귀납적 편향이 거의 없음

시계열 데이터

- 시간적 관계를 잘 표현함
- sequential 편향을 가진 순환 신경망 모델 많이 사용함

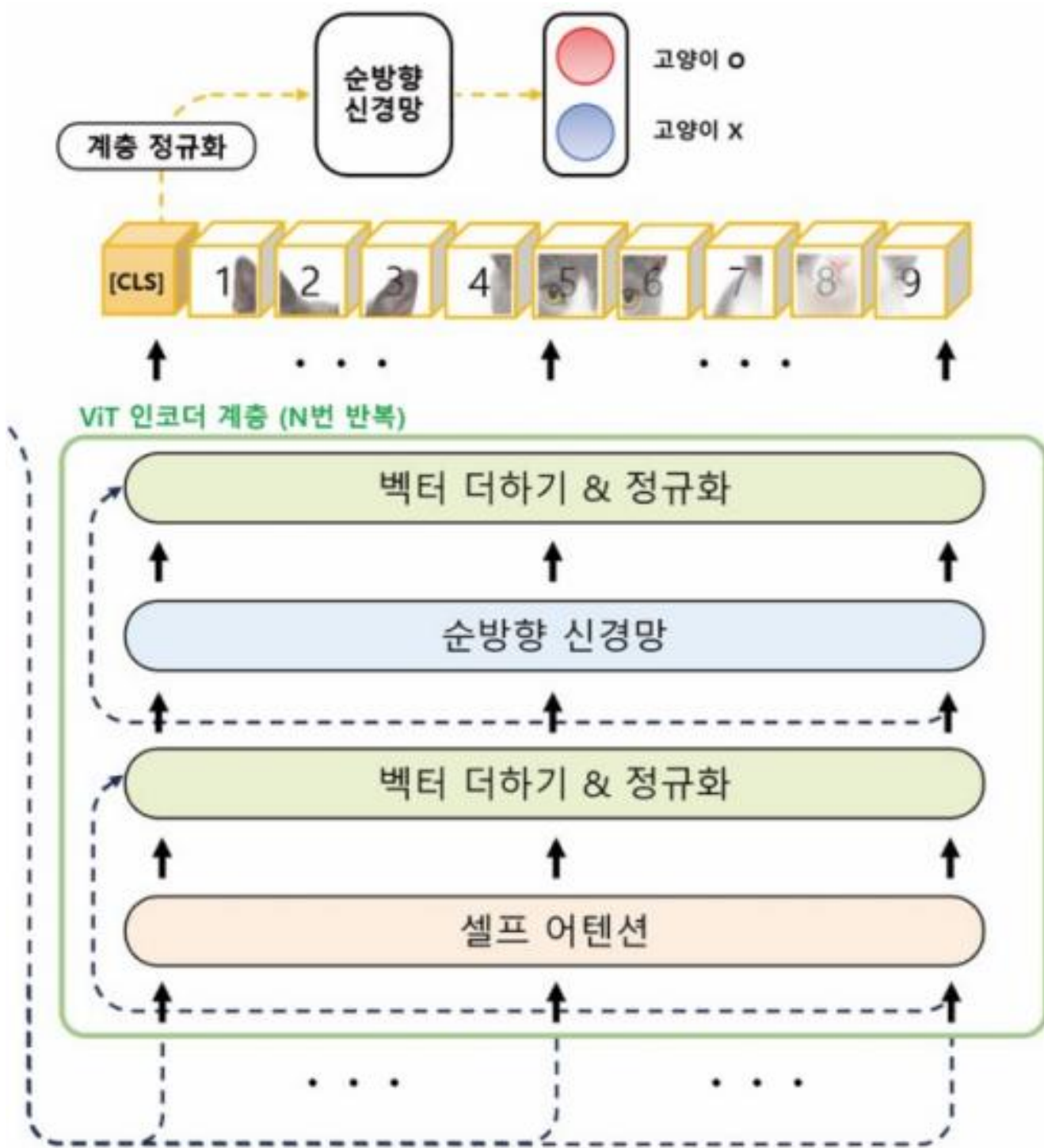
순환 신경망

- 이전 시간의 상태를 기억하고, 현재 입력과 결합해서 다음 상태 예측
- 순차적인 특징을 강조해서 시계열 데이터에서 좋은 성능
- 단점: 특정 관계 편향이 강할수록 다른 다양한 관계 표현이 어려워서 귀납적 편향이 약한 모델을 선호하게 됨

ViT 모델

- 입력 데이터의 다양한 쿼리, 키, 값의 임베딩 형태 일반화된 관계를 학습
- 귀납적 편향이 거의 없고, 대용량 데이터에 대해 이미지 특징들의 관계를 잘 학습함

#04 ViT 모델



ViT 모델 구조

ViT 모델

1. 입력 이미지를 트랜스포머 구조에 맞게 일정한 크기로 패치를 나눔
2. 각 패치를 벡터 형태로 변환 (패치 임베딩, 인코더)
3. 패치 임베딩, 인코더 계층을 통해 이미지 추출
4. 작업(분류, 회귀)에 맞는 출력값으로 변환해 사용

#04 ViT 모델

패치 임베딩

입력 이미지를 작은 패치로 분할하는 과정
작은 패치로 분할하려면? 이미지 크기를 맞추는
전처리 필요 (정방향으로)

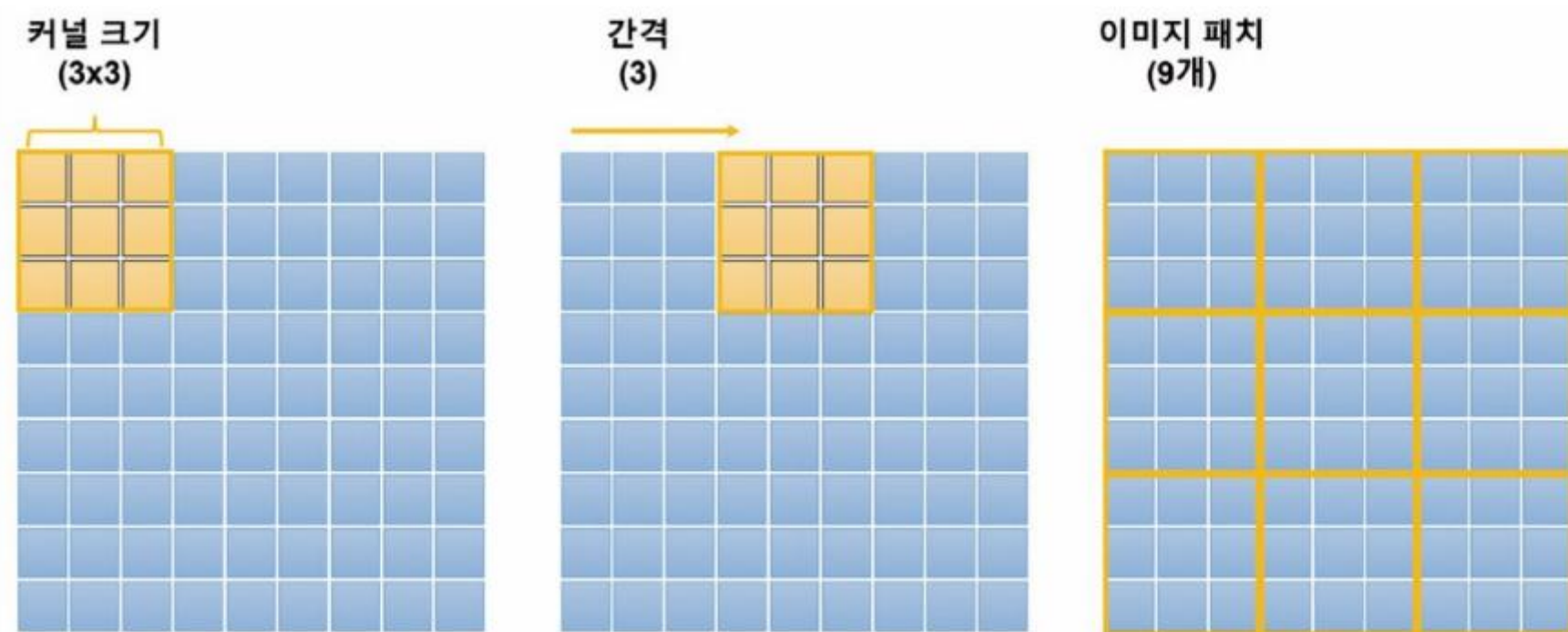


그림 10.5 이미지 패치 생성 과정

Kernel size: 패치 크기
Stride(간격): 패치가 이동하는 폭
패치 식 사진
둘다 하이퍼 파라미터로 정의됨

수식 10.1 패치 계산 과정

$$patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9 - 3}{3} + 1 = 3$$

#04 ViT 모델

1. 분할된 이미지 패치들을 배열 형태로 나타낼 때 원 → 오, 위 → 아래 순서로 배열함.
2. 이 배열 가장 왼쪽에 분류 토큰([CLS]) 추가함
3. 위치 임베딩 사용해서 인접한 패치 간의 관계 학습
4. 계층 정규화
5. 패치 임베딩 생성
6. 입력 이미지를 작은 패치로 나눠 처리 ⇒ GPU 메모리의 한계를 극복, 더 큰 이미지 처리 가능

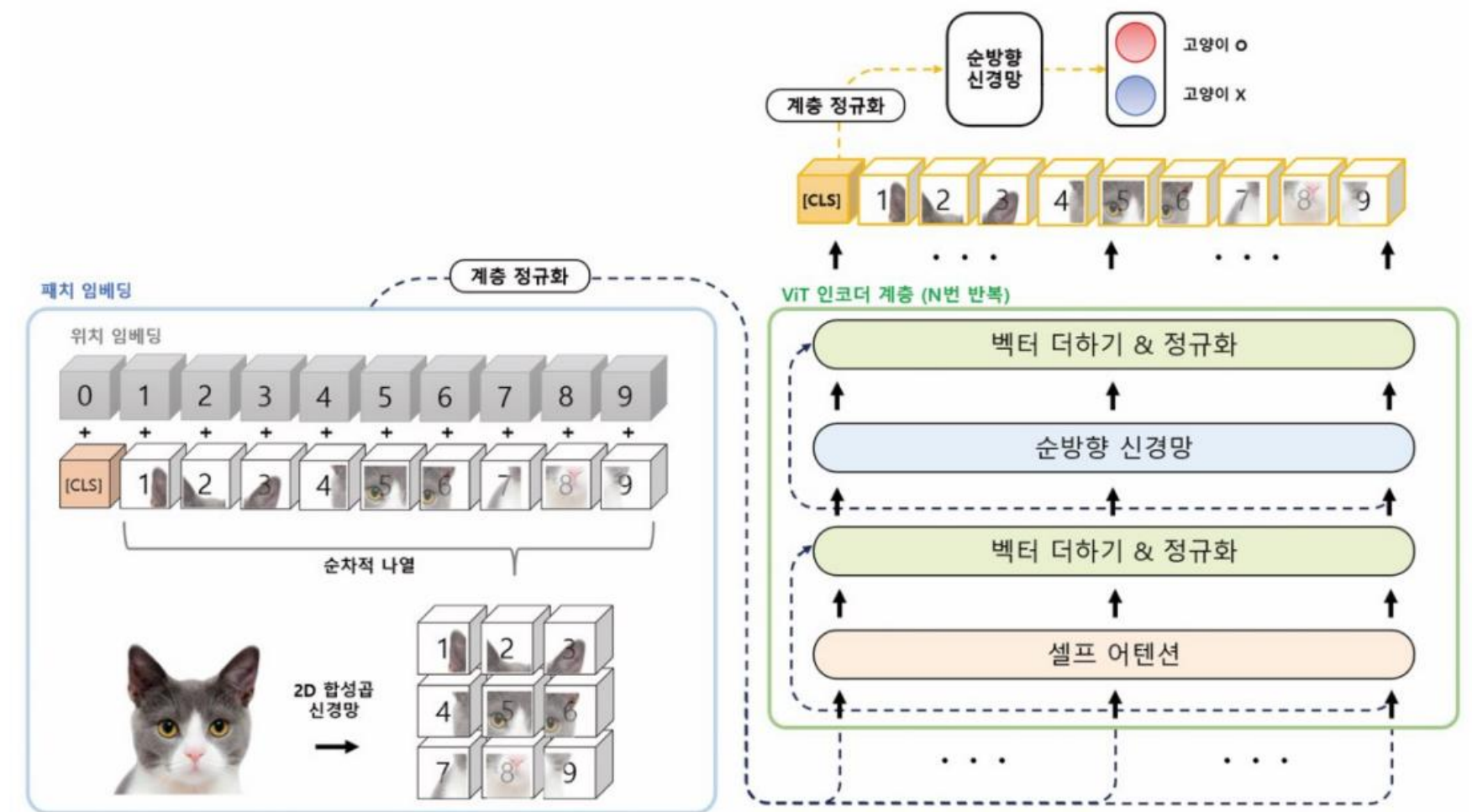


그림 10.4 ViT 모델 구조

- * 분류 토큰: 전체 이미지를 대표하는 벡터. 특정 문제를 예측하는데 사용
- * 위치 임베딩: 패치의 위치 정보를 임베딩 벡터로 변환, 기존 이미지 패치 벡터들과 더함

#05 인코더 계층

인코더 계층

- ViT 모델은 멀티 헤드 어텐션과 학습 과정이 동일함
- 과정
 1. 이미지를 일정한 크기의 패치로 나눔
 2. 각 패치를 벡터로 변환
 3. 인코더 계층에 입력으로 전달
 4. 다양한 쿼리, 키, 값 임베딩의 관계 학습
 5. N개의 인코더 레이어를 반복적으로 적용
 6. 마지막 레이어에서 특별한 패치의 특징 벡터 추출 (분류 토큰)

*분류 토큰 벡터: 이미지 데이터를 잘 표현하는 특징 벡터로 다양한 이미지 분류 및 검색 문제 해결하는데 사용함

ViT 모델

- 시퀀셜 산출물을 모두 사용하는 것이 아니라 전체 이미지의 특징을 잘 표현하는 분류 토큰 벡터만 사용
- 분류 토큰 벡터를 순방향 신경망(완전 연결 계층)에 연결 → 고양이인지 아닌지를 분류 → ViT 모델 학습

#06 모델 실습

```
1 from itertools import chain
2 from collections import defaultdict
3 from torch.utils.data import Subset
4 from torchvision import datasets
5 def subset_sampler(dataset, classes, max_len):
6     target_idx = defaultdict(list)
7     for idx, label in enumerate(dataset.train_labels):
8         target_idx[int(label)].append(idx)
9
10    indices = list(
11        chain.from_iterable(
12            [target_idx[idx][:max_len] for idx in range(len(classes))]
13        )
14    )
15    return Subset(dataset, indices)
16
17
18 train_dataset = datasets.FashionMNIST(root="../datasets", download=True, train=True)
19 test_dataset = datasets.FashionMNIST(root="../datasets", download=True, train=False)
20
21 classes = train_dataset.classes
22 class_to_idx = train_dataset.class_to_idx
23
24 print(classes)
25 print(class_to_idx)
26
27 subset_train_dataset = subset_sampler(
28     dataset=train_dataset, classes=train_dataset.classes, max_len=1000
29 )
30 subset_test_dataset = subset_sampler(
31     dataset=test_dataset, classes=test_dataset.classes, max_len=100
32 )
33
34 print(f"Training Data Size : {len(subset_train_dataset)}")
35 print(f"Testing Data Size : {len(subset_test_dataset)}")
36 print(train_dataset[0])
```

```
... 100%|██████████| 26.4M/26.4M [00:01<00:00, 17.2MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 295kB/s]
100%|██████████| 4.42M/4.42M [00:00<00:00, 5.05MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 18.2MB/s]
/usr/local/lib/python3.12/dist-packages/torchvision/datasets/mnist.py:66: UserWarning: train_labels has been renamed targets
  warnings.warn("train_labels has been renamed targets")
['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
{'T-shirt/top': 0, 'Trouser': 1, 'Pullover': 2, 'Dress': 3, 'Coat': 4, 'Sandal': 5, 'Shirt': 6, 'Sneaker': 7, 'Bag': 8, 'Ankle boot': 9}
Training Data Size : 10000
Testing Data Size : 1000
(<PIL.Image.Image image mode=L size=28x28 at 0x7A54037D4410>, 9)
```

EWCHA
EUROPEAN

```
... /usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
preprocessor_config.json: 100%  160/160 [00:00<00:00, 5.31kB/s]

config.json: 100%  502/502 [00:00<00:00, 13.9kB/s]

Fast image processor class <class 'transformers.models.vit.image_processing_vit_fast.ViTImageProcessorFast'> is available for
size : {'height': 224, 'width': 224}
mean : [0.5, 0.5, 0.5]
std : [0.5, 0.5, 0.5]
```

#06 모델 실습

```
1 from torch.utils.data import DataLoader
2
3
4 def collator(data, transform):
5     images, labels = zip(*data)
6     pixel_values = torch.stack([transform(image) for image in images])
7     labels = torch.tensor([label for label in labels])
8     return {"pixel_values": pixel_values, "labels": labels}
9
10
11 train_dataloader = DataLoader(
12     subset_train_dataset,
13     batch_size=32,
14     shuffle=True,
15     collate_fn=lambda x: collator(x, transform),
16     drop_last=True
17 )
18 valid_dataloader = DataLoader(
19     subset_test_dataset,
20     batch_size=4,
21     shuffle=True,
22     collate_fn=lambda x: collator(x, transform),
23     drop_last=True
24 )
25
26 batch = next(iter(train_dataloader))
27 for key, value in batch.items():
28     print(f"{key} : {value.shape}")
```

```
pixel_values : torch.Size([32, 3, 224, 224])
labels : torch.Size([32])
```

#06 모델 실습

```
1 from transformers import ViTForImageClassification
2
3
4 model = ViTForImageClassification.from_pretrained(
5     pretrained_model_name_or_path="google/vit-base-patch16-224-in21k",
6     num_labels=len(classes),
7     id2label={idx: label for label, idx in class_to_idx.items()},
8     label2id=class_to_idx,
9     ignore_mismatched_sizes=True
10 )
11
12 print(model.classifier)
```

model.safetensors: 100%  346M/346M [00:05<00:00, 84.8MB/s]

Loading weights: 100%  198/198 [00:00<00:00, 275.53it/s, Materializing param=vit.layernorm.weight]

ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224-in21k

Key	Status
pooler.dense.weight	UNEXPECTED
pooler.dense.bias	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Linear(in_features=768, out_features=10, bias=True)

#06 모델 실습

```
1 print(model.vit.embeddings)
2
3 batch = next(iter(train_dataloader))
4 print("image shape :", batch["pixel_values"].shape)
5 print("patch embeddings shape :",
6       model.vit.embeddings.patch_embeddings(batch["pixel_values"]).shape)
7
8 print("[CLS] + patch embeddings shape :",
9       model.vit.embeddings(batch["pixel_values"]).shape)
10 )
```

```
... ViTEmbeddings(
  (patch_embeddings): ViTPatchEmbeddings(
    (projection): Conv2d(3, 768, kernel_size=(16, 16), stride=(16, 16))
  )
  (dropout): Dropout(p=0.0, inplace=False)
)
image shape : torch.Size([32, 3, 224, 224])
patch embeddings shape : torch.Size([32, 196, 768])
[CLS] + patch embeddings shape : torch.Size([32, 197, 768])
```

#06 모델 실습

```
1 from transformers import TrainingArguments
2
3
4 args = TrainingArguments(
5     output_dir="../models/ViT-FashionMNIST",
6     save_strategy="epoch",
7     eval_strategy="epoch",
8     learning_rate=1e-5,
9     per_device_train_batch_size=16,
10    per_device_eval_batch_size=16,
11    num_train_epochs=3,
12    weight_decay=0.001,
13    load_best_model_at_end=True,
14    metric_for_best_model="f1",
15    logging_dir="logs",
16    logging_steps=125,
17    remove_unused_columns=False,
18    seed=7
19 )
```

```
1 import evaluate
2 import numpy as np
3
4
5 def compute_metrics(eval_pred):
6     metric = evaluate.load("f1")
7     predictions, labels = eval_pred
8     predictions = np.argmax(predictions, axis=1)
9     macro_f1 = metric.compute(
10         predictions=predictions, references=labels, average="macro"
11     )
12     return macro_f1
```

#06 모델 실습

```
1 import torch
2 import evaluate
3 import numpy as np
4 from itertools import chain
5 from collections import defaultdict
6 from torch.utils.data import Subset
7 from torchvision import datasets
8 from torchvision import transforms
9 from transformers import AutoImageProcessor
10 from transformers import ViTForImageClassification
11 from transformers import TrainingArguments, Trainer
12
13
14 def subset_sampler(dataset, classes, max_len):
15     target_idx = defaultdict(list)
16     for idx, label in enumerate(dataset.train_labels):
17         target_idx[int(label)].append(idx)
18
19     indices = list(
20         chain.from_iterable(
21             [target_idx[idx][:max_len] for idx in classes]
22         )
23     )
24     return Subset(dataset, indices)
25
26
27 def model_init(classes, class_to_idx):
28     model = ViTForImageClassification.from_pretrained(
29         pretrained_model_name_or_path="google/vit-base-patch16-224",
30         num_labels=len(classes),
31         id2label={idx: label for label, idx in class_to_idx.items()},
32         label2id={label: idx for label, idx in class_to_idx.items()}
33     )
34     return model
```

```
37 def collator(data, transform):
38     images, labels = zip(*data)
39     pixel_values = torch.stack([transform(image) for image in images])
40     labels = torch.tensor([label for label in labels])
41     return {"pixel_values": pixel_values, "labels": labels}
42
43
44 def compute_metrics(eval_pred):
45     metric = evaluate.load("f1")
46     predictions, labels = eval_pred
47     predictions = np.argmax(predictions, axis=1)
48     macro_f1 = metric.compute(
49         predictions=predictions, references=labels, average="macro"
50     )
51     return macro_f1
52
53
54 train_dataset = datasets.FashionMNIST(root= "../datasets", download=True, transform=transform)
55 test_dataset = datasets.FashionMNIST(root= "../datasets", download=True, transform=transform)
```

You should probably train this model on a down-stream task to be able to use it for inference.

[1875/1875 17:31, Epoch 3/3]

Epoch	Training Loss	Validation Loss	F1
1	0.701000	0.632113	0.887223
2	0.477200	0.473497	0.907536
3	0.417100	0.426318	0.924432

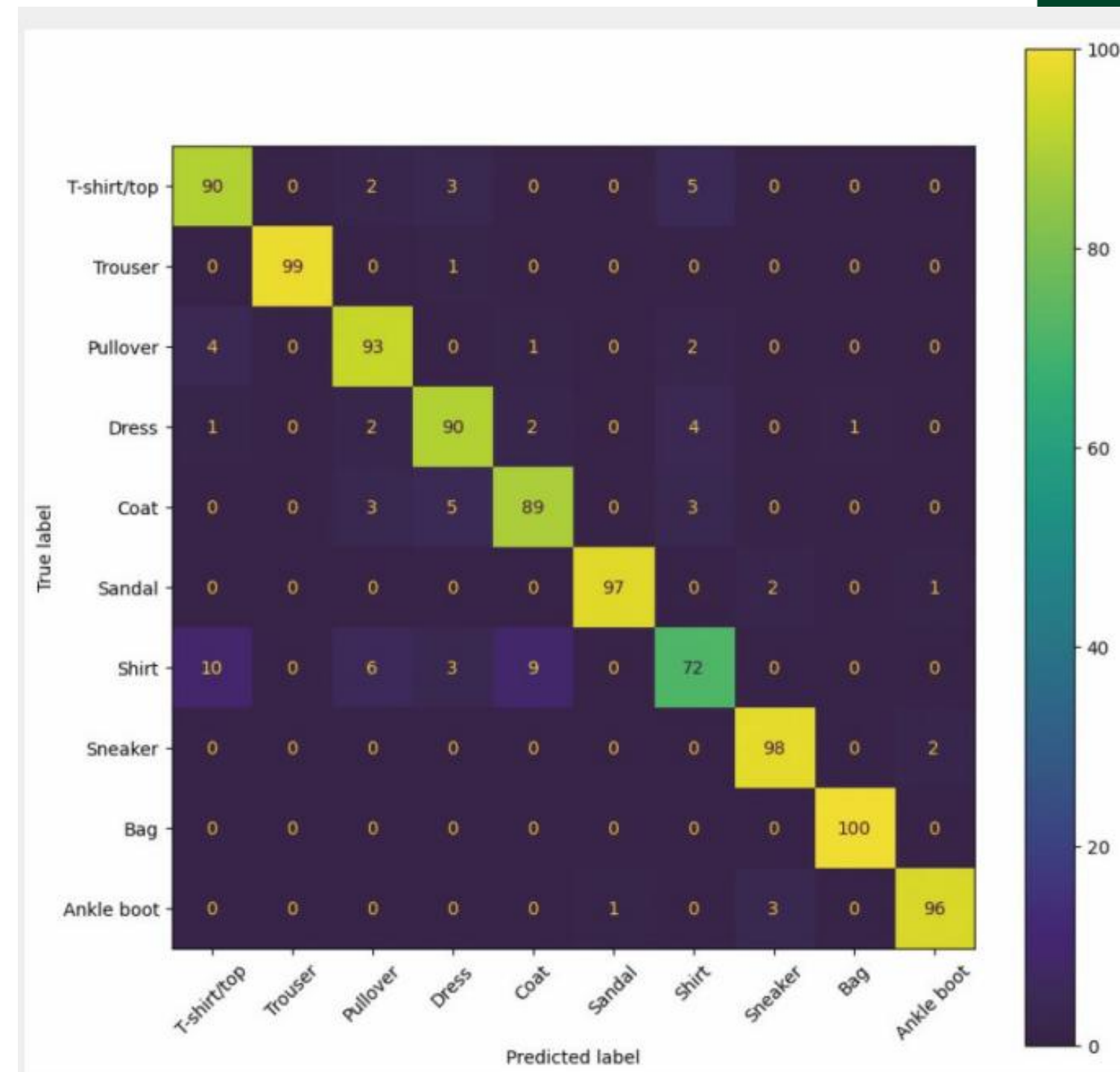
Downloading builder script:  6.79k/? [00:00<00:00, 636kB/s]

TrainOutput(global_step=1875, training_loss=0.6974474385579427, metrics={'train_runtime': 1053.7617, 'train_samples_per_second': 28.469, 'train_steps_per_second': 1.779, 'total_flos': 2.32492637712384e+18, 'train_loss': 0.6974474385579427, 'epoch': 3.0})

```
71 transform = transforms.Compose(
72     [
73         transforms.ToTensor(),
74         transforms.Resize(
75             size=(
76                 image_processor.size["height"],
77                 image_processor.size["width"]
78             )
79         ),
80         transforms.Lambda(
81             lambda x: torch.cat([x, x, x], 0)
82         ),
83         transforms.Normalize(
84             mean=image_processor.image_mean,
85             std=image_processor.image_std
86         )
87     ]
88 )
89
90 args = TrainingArguments(
91     output_dir= "../models/ViT-FashionMNIST",
92     save_strategy="epoch",
93     eval_strategy="epoch",
94     learning_rate=1e-5,
95     per_device_train_batch_size=16,
96     per_device_eval_batch_size=16,
97     num_train_epochs=3,
98     weight_decay=0.001,
99     load_best_model_at_end=True,
100     metric_for_best_model="f1",
101     logging_dir="logs",
102     logging_steps=125,
103     remove_unused_columns=False,
104     seed=7,
105     report_to="none"
106 )
107
108 trainer = Trainer(
109     model_init=lambda x: model_init(classes, class_to_idx),
110     args=args,
111     train_dataset=subset_train_dataset,
112     eval_dataset=subset_test_dataset,
113     data_collator=lambda x: collator(x, transform),
114     compute_metrics=compute_metrics,
115     tokenizer=image_processor,
116 )
117 trainer.train()
```

#06 모델 실습

```
1 import matplotlib.pyplot as plt
2 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
3
4
5 outputs = trainer.predict(subset_test_dataset)
6 print(outputs)
7
8 y_true = outputs.label_ids
9 y_pred = outputs.predictions.argmax(1)
10
11 labels = list(classes)
12 matrix = confusion_matrix(y_true, y_pred)
13 display = ConfusionMatrixDisplay(confusion_matrix=matrix, display_labels=labels)
14 _, ax = plt.subplots(figsize=(10, 10))
15 display.plot(xticks_rotation=45, ax=ax)
16 plt.show()
```



#01 Swin Transformer vs ViT

ViT : 공간정보 완전히 보존 x, 이미지 패치 수 증가 -
> 학습 데이터 크기 증가 -> 계산량 증가
Solution : Local window

물체의 크기나 해상도를 계층적으로 학습
시프트 윈도우 기술 도입하여 입력 이미지 일정한
크기의 패치로 분할,
맵 이동시켜 다음 계층에 사용해 패치의 위치를 더
유연하게 다룸

회색격자 : 패치
붉은 격자 : 로컬윈도

그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

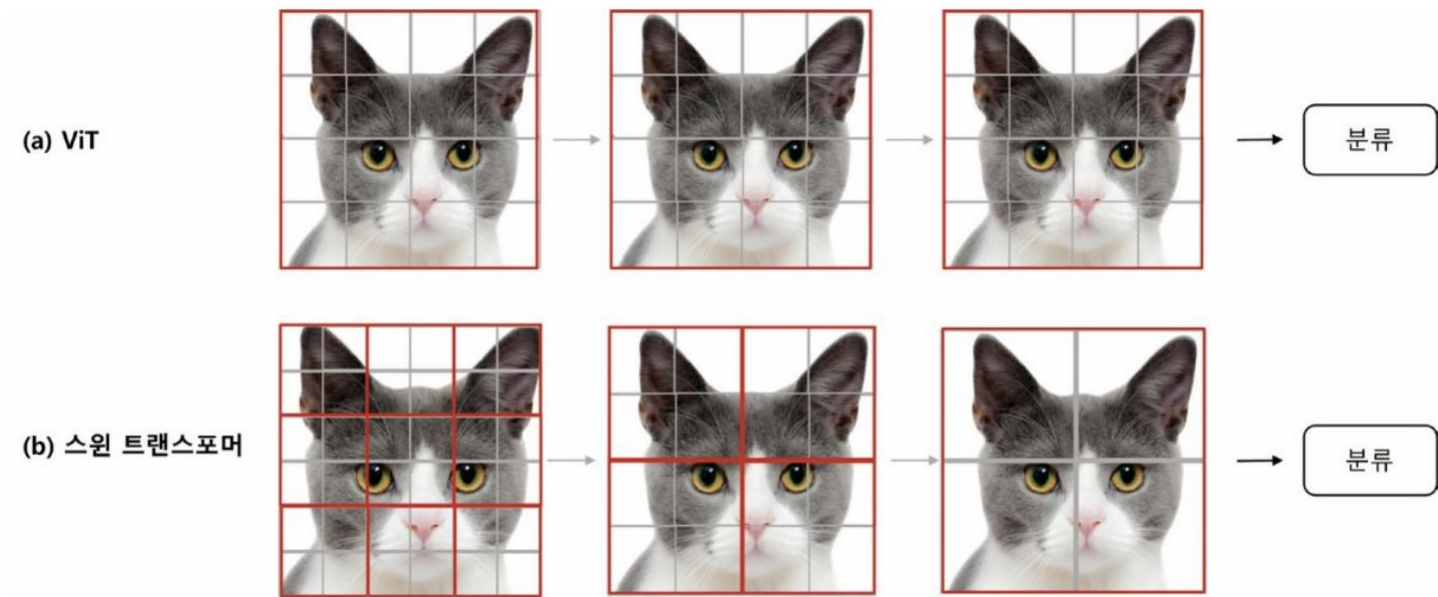


그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

표 10.4 ViT와 스윈 트랜스포머 비교

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도우 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

#02 Swin 구조

입력 이미지 일정한 크기의 패치로 분할 -> 벡터화
분할된 패치를 기반으로 스윈 트랜스포머 블록 구성
블록 : 어텐션 계층, 계층 정규화, 다중퍼셉트론 등의 패치 간 상호작용

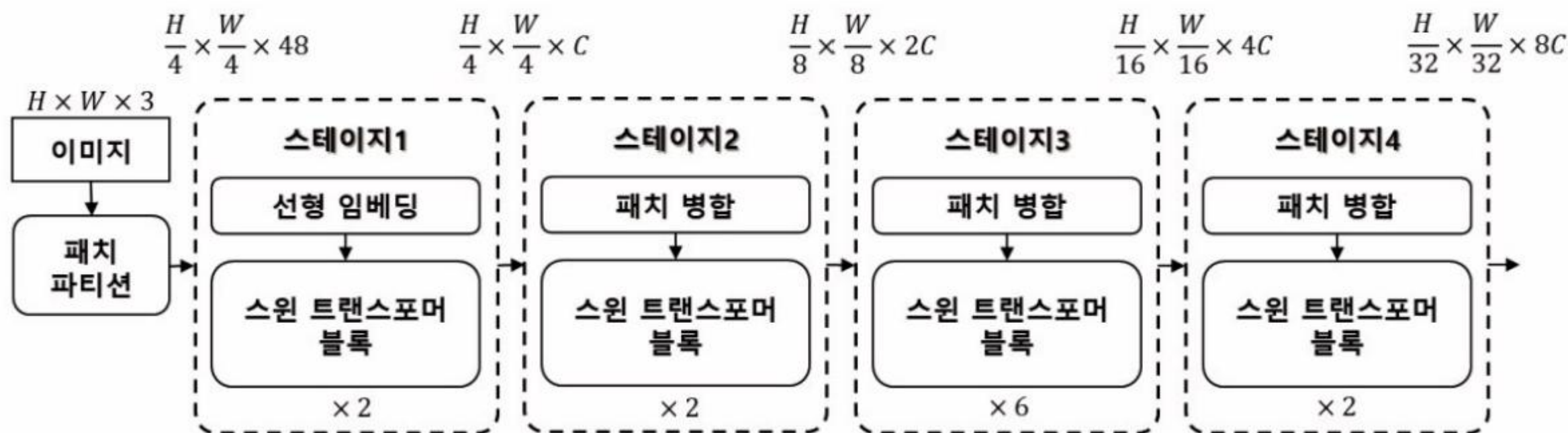


그림 10.7 스윈 트랜스포머 구조

#02 Swin 구조

패치 파티션 : 입력 이미지를 작은 사각형 패치로 분할해 처리하기

패치는 트랜스포머의 입력으로 사용됨

패치 병합 : 저차원으로 축소

$[C, H, W] \rightarrow [2C, H/2, W/2]$ 로 재정렬

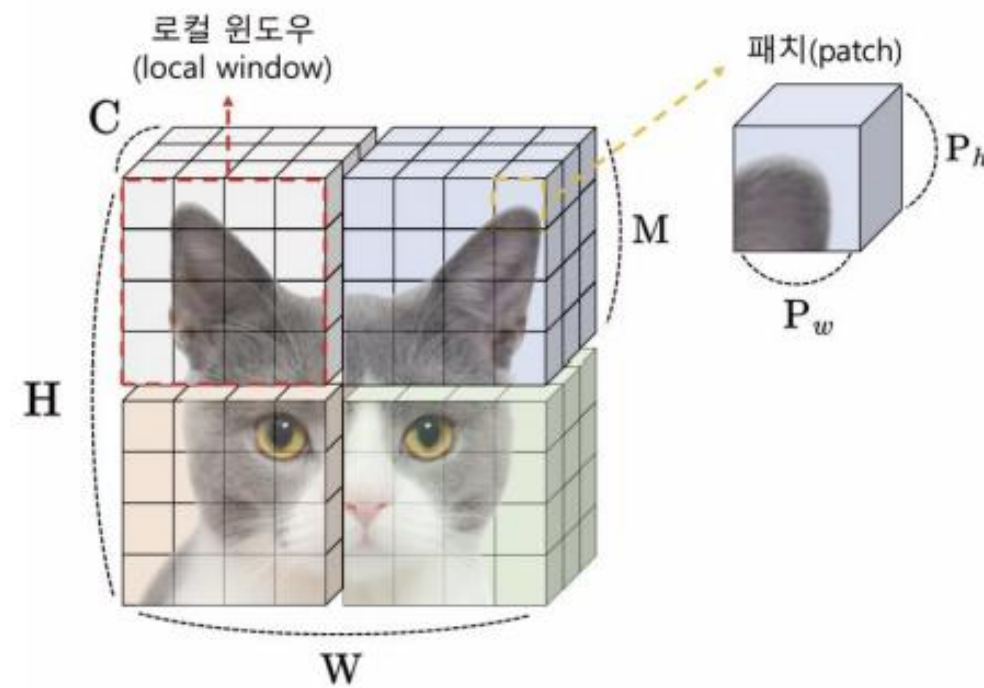


그림 10.8 패치 파티션 구조

#02 Swin 구조

W-MSA : 효율적인 배치 계산 사용
SW-MSA : 순환시프트, 어텐션 마스크 사용

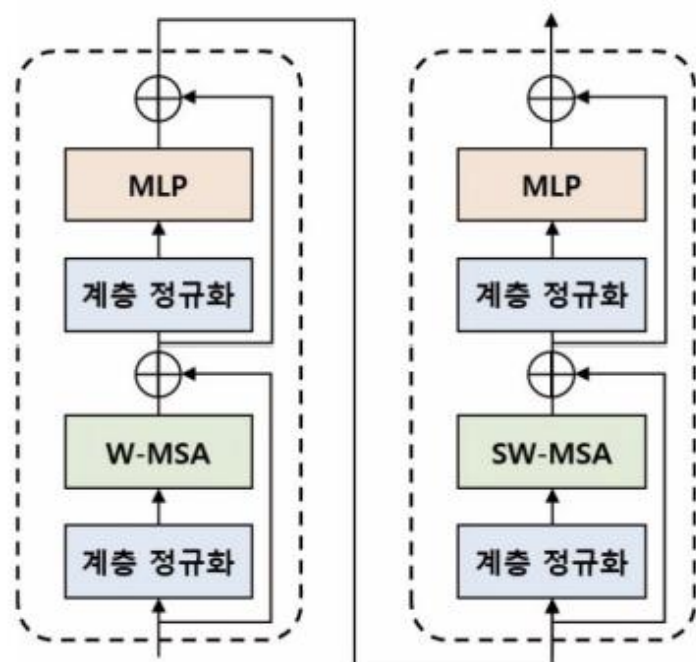


그림 10.9 스윈 트랜스포머 블록 구조

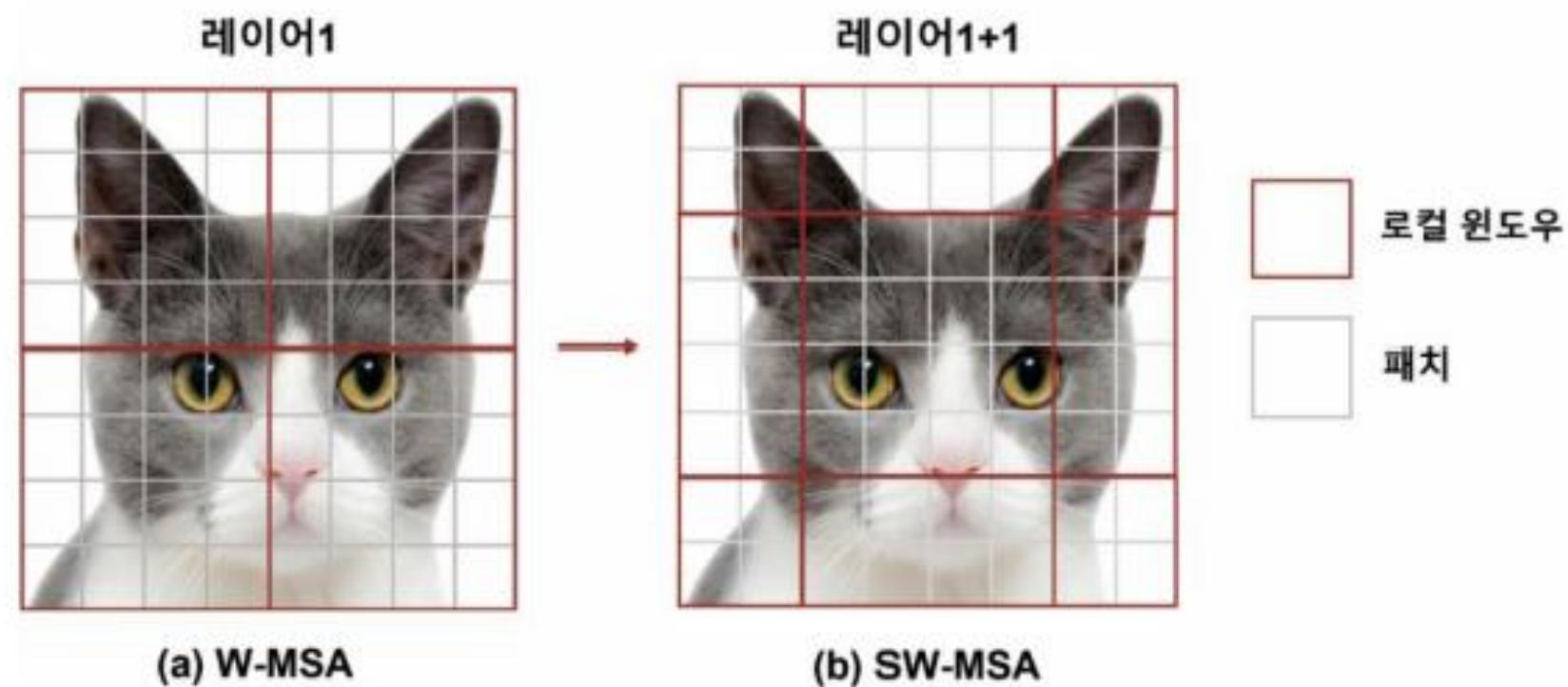


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

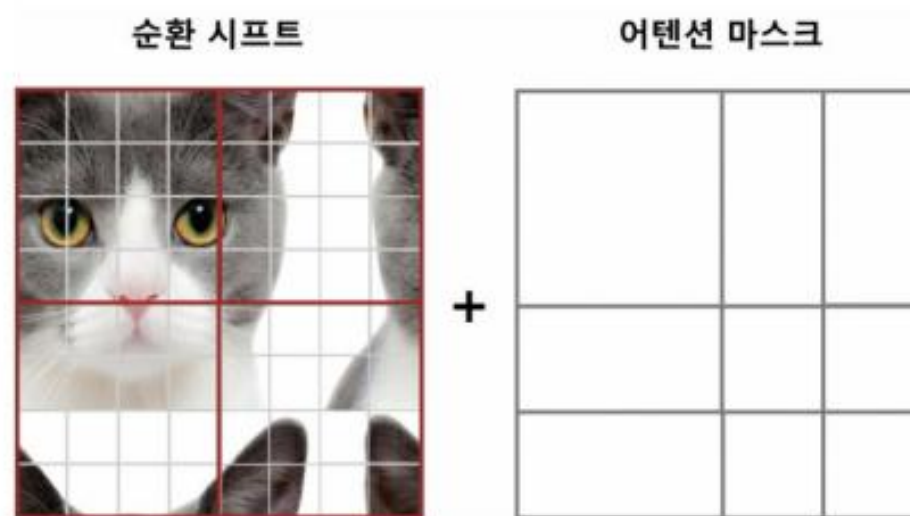


그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

#02 Swin 구조

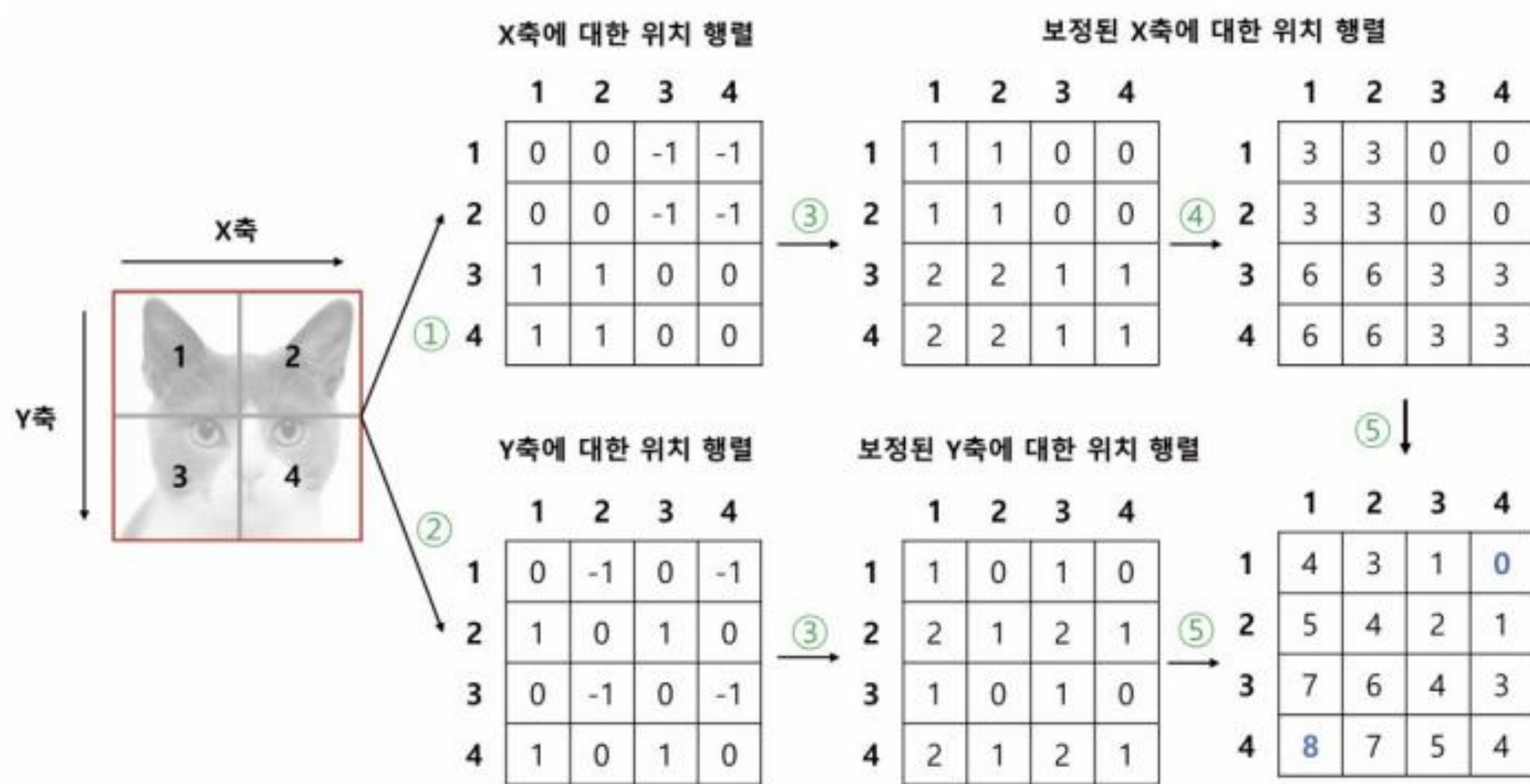


그림 10.13 상대적 위치 편향 계산 과정

#02 Swin 구조

```
[1]
✓ 4초 ▶ 1 |import torch
2
3
4 window_size = 2
5 coords_h = torch.arange(window_size)
6 coords_w = torch.arange(window_size)
7 coords = torch.stack(torch.meshgrid([coords_h, coords_w], indexing="ij"))
8 coords_flatten = torch.flatten(coords, 1)
9 relative_coors = coords_flatten[:, :, None] - coords_flatten[:, None, :]
10
11 print(relative_coors)
12 print(relative_coors.shape)

... tensor([[[ 0,  0, -1, -1],
             [ 0,  0, -1, -1],
             [ 1,  1,  0,  0],
             [ 1,  1,  0,  0]],

            [[ 0, -1,  0, -1],
             [ 1,  0,  1,  0],
             [ 0, -1,  0, -1],
             [ 1,  0,  1,  0]]])
torch.Size([2, 4, 4])
```

#02 Swin 구조

```
[2]
0초
1 x_coors = relative_coors[0, :, :]
2 y_coors = relative_coors[1, :, :]
3
4 x_coors += window_size - 1 # X축에 대한 ③번 연산 과정
5 y_coors += window_size - 1 # Y축에 대한 ③번 연산 과정
6 x_coors *= 2 * window_size - 1 # ④번 연산 과정
7 print(f"X축에 대한 행렬:{x_coors}\n")
8 print(f"Y축에 대한 행렬:{y_coors}\n")
9
10 relative_position_index = x_coors + y_coors # ⑤번 연산 과정
11 print(f"X, Y축에 대한 위치 행렬:{relative_position_index}")
```

▼ X축에 대한 행렬:

```
tensor([[3, 3, 0, 0],
        [3, 3, 0, 0],
        [6, 6, 3, 3],
        [6, 6, 3, 3]])
```

Y축에 대한 행렬:

```
tensor([[1, 0, 1, 0],
        [2, 1, 2, 1],
        [1, 0, 1, 0],
        [2, 1, 2, 1]])
```

X, Y축에 대한 위치 행렬:

```
tensor([[4, 3, 1, 0],
        [5, 4, 2, 1],
        [7, 6, 4, 3],
        [8, 7, 5, 4]])
```

수식 10.2 상대적 위치 편향을 고려한 셀프 어텐션

$$\text{Attention}(Q, K, V) = \text{SoftMax}(QK^T / \sqrt{d} + B)V$$

#03 모델 실습

```
1 from transformers import SwinForImageClassification
2
3
4 model = SwinForImageClassification.from_pretrained(
5     pretrained_model_name_or_path="microsoft/swin-tiny-patch4-window7-224",
6     num_labels=len(train_dataset.classes),
7     id2label={idx: label for label, idx in train_dataset.class_to_idx.items()},
8     label2id=train_dataset.class_to_idx,
9     ignore_mismatched_sizes=True
10 )
11
12 for main_name, main_module in model.named_children():
13     print(main_name)
14     for sub_name, sub_module in main_module.named_children():
15         print("└", sub_name)
16         for ssub_name, ssub_module in sub_module.named_children():
17             print("│ └", ssub_name)
18             for sssub_name, sssub_module in ssub_module.named_children():
19                 if sssub_name == "projection":
20                     print("│ │ └", sssub_name, sssub_module)
21                 else:
22                     print("│ │ └", sssub_name)
```

Notes:

- MISMATCH :ckpt weights were loaded, but they did not match the original empty weight shapes.

```
swin
└ embeddings
  │ └ patch_embeddings
  │   └ projection Conv2d(3, 96, kernel_size=(4, 4), stride=(4, 4))
  │   └ norm
  │   └ dropout
└ encoder
  │ └ layers
  │   │ └ 0
  │   │ └ 1
  │   │ └ 2
  │   │ └ 3
└ layernorm
└ pooler
classifier
```


#03 모델 실습

```
1 batch = next(iter(train_dataloader))
2 print("이미지 차원 :", batch["pixel_values"].shape)
3
4 patch_emb_output, shape = model.swin.embeddings.patch_embeddings(batch["pixel_values"])
5 print("모듈:", model.swin.embeddings.patch_embeddings)
6 print("패치 임베딩 차원 :", patch_emb_output.shape)
```

```
이미지 차원 : torch.Size([32, 3, 224, 224])
모듈: SwinPatchEmbeddings(
  (projection): Conv2d(3, 96, kernel_size=(4, 4), stride=(4, 4))
)
패치 임베딩 차원 : torch.Size([32, 3136, 96])
```

#03 모델 실습

```
[6] 0초 ▶ 1 for main_name, main_module in model.swin.encoder.layers[0].named_children():
2     print(main_name)
3     for sub_name, sub_module in main_module.named_children():
4         print("  L", sub_name)
5         for ssub_name, ssub_module in sub_module.named_children():
6             print("    |  L", ssub_name)
```

▼

```
*** blocks
  L 0
  |  L layernorm_before
  |  L attention
  |  L drop_path
  |  L layernorm_after
  |  L intermediate
  |  L output
  L 1
  |  L layernorm_before
  |  L attention
  |  L drop_path
  |  L layernorm_after
  |  L intermediate
  |  L output
downsample
  L reduction
  L norm
```

#03 모델 실습

```
[7]
✓ 0초
1 print(model.swin.encoder.layers[0].blocks[0])

SwinLayer(
  (layernorm_before): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
  (attention): SwinAttention(
    (self): SwinSelfAttention(
      (query): Linear(in_features=96, out_features=96, bias=True)
      (key): Linear(in_features=96, out_features=96, bias=True)
      (value): Linear(in_features=96, out_features=96, bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
    )
    (output): SwinSelfOutput(
      (dense): Linear(in_features=96, out_features=96, bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
    )
  )
  (drop_path): Identity()
  (layernorm_after): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
  (intermediate): SwinIntermediate(
    (dense): Linear(in_features=96, out_features=384, bias=True)
    (intermediate_act_fn): GELUActivation()
  )
  (output): SwinOutput(
    (dense): Linear(in_features=384, out_features=96, bias=True)
    (dropout): Dropout(p=0.0, inplace=False)
  )
)
```

#03 모델 실습

```
[8]
✓ 7초
1 print("패치 임베딩 차원 :", patch_emb_output.shape)
2
3 W_MSA = model.swin.encoder.layers[0].blocks[0]
4 SW_MSA = model.swin.encoder.layers[0].blocks[1]
5
6 W_MSA_output = W_MSA(patch_emb_output, W_MSA.input_resolution)[0]
7 SW_MSA_output = SW_MSA(W_MSA_output, SW_MSA.input_resolution)[0]
8
9 print("W-MSA 결과 차원 :", W_MSA_output.shape)
10 print("SW-MSA 결과 차원 :", SW_MSA_output.shape)
```

✓

패치 임베딩 차원 : torch.Size([32, 3136, 96])
W-MSA 결과 차원 : torch.Size([32, 3136, 96])
SW-MSA 결과 차원 : torch.Size([32, 3136, 96])

```
[9]
✓ 0초
1 patch_merge = model.swin.encoder.layers[0].downsample
2 print("patch_merge 모듈 :", patch_merge)
3
4 output = patch_merge(SW_MSA_output, patch_merge.input_resolution)
5 print("patch_merge 결과 차원 :", output.shape)
```

✓

patch_merge 모듈 : SwinPatchMerging(
 (reduction): Linear(in_features=384, out_features=192, bias=False)
 (norm): LayerNorm((384,), eps=1e-05, elementwise_affine=True)
)
patch_merge 결과 차원 : torch.Size([32, 784, 192])

#03 모델 실습

ViT 모델 미세 조정 방식과 동일하게 훈련자 클래스(Trainer)를 통해 스윈 트랜스포머 모델의 학습을 수행한다. 사전 학습된 모델과 출력 디렉터리만 변경된다.

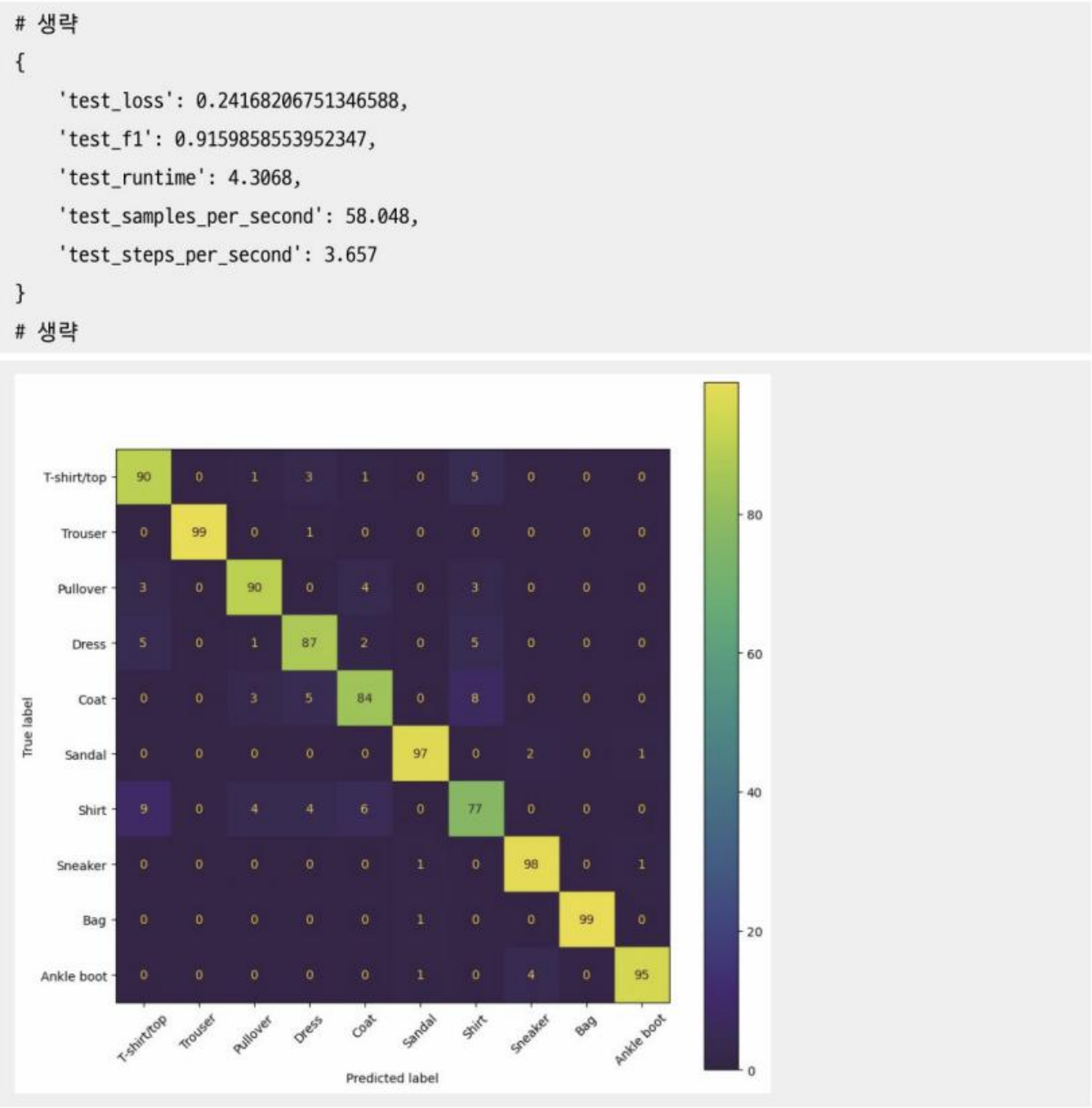
학습이 완료되면 Swin-FashionMNIST 디렉터리에 체크포인트마다 학습 결과, 성능 지표, 하이퍼파라미터 등이 저장된다. 다음 표 10.3은 스윈 트랜스포머 모델의 학습 결과를 정리한 것이다.

표 10.3 스윈 트랜스포머 학습 결과

Epoch	Training Loss	Validation Loss	F1
1	0.3720	0.3046	0.8985
2	0.2579	0.2688	0.9138
3	0.2607	0.2416	0.9159

테스트 데이터세트로 추론했을 때 스윈 트랜스포머 모델은 약 91.6%의 분류 정확도를 갖는다. 앞선 ViT 모델의 학습 결과와 비교했을 때 F1 성능 지표는 거의 동일하지만, 검증 손실값이 0.4357에서 0.2416으로 대폭 낮아졌다. 다음 출력 결과는 예제 10.9 ViT 모델 성능 평가를 활용해 혼동 행렬로 모델의 성능을 평가한 것이다.

출력 결과



스윈 트랜스포머 모델과 ViT 모델의 평가 결과를 비교해 보면 스윈 트랜스포머 모델이 ‘셔츠’ 이미지를 더 높은 확률로 예측하며, ‘티셔츠’, ‘코트’로 오분류하는 문제가 완화된 것을 확인할 수 있다. 또한 스윈 트랜스포머 모델의 수렴 속도가 더 빠른 것을 확인할 수 있다.

#10.3 CvT

CvT: 합성곱 신경망 구조에서 활용하는 계층형 구조를 ViT에 적용한 모델

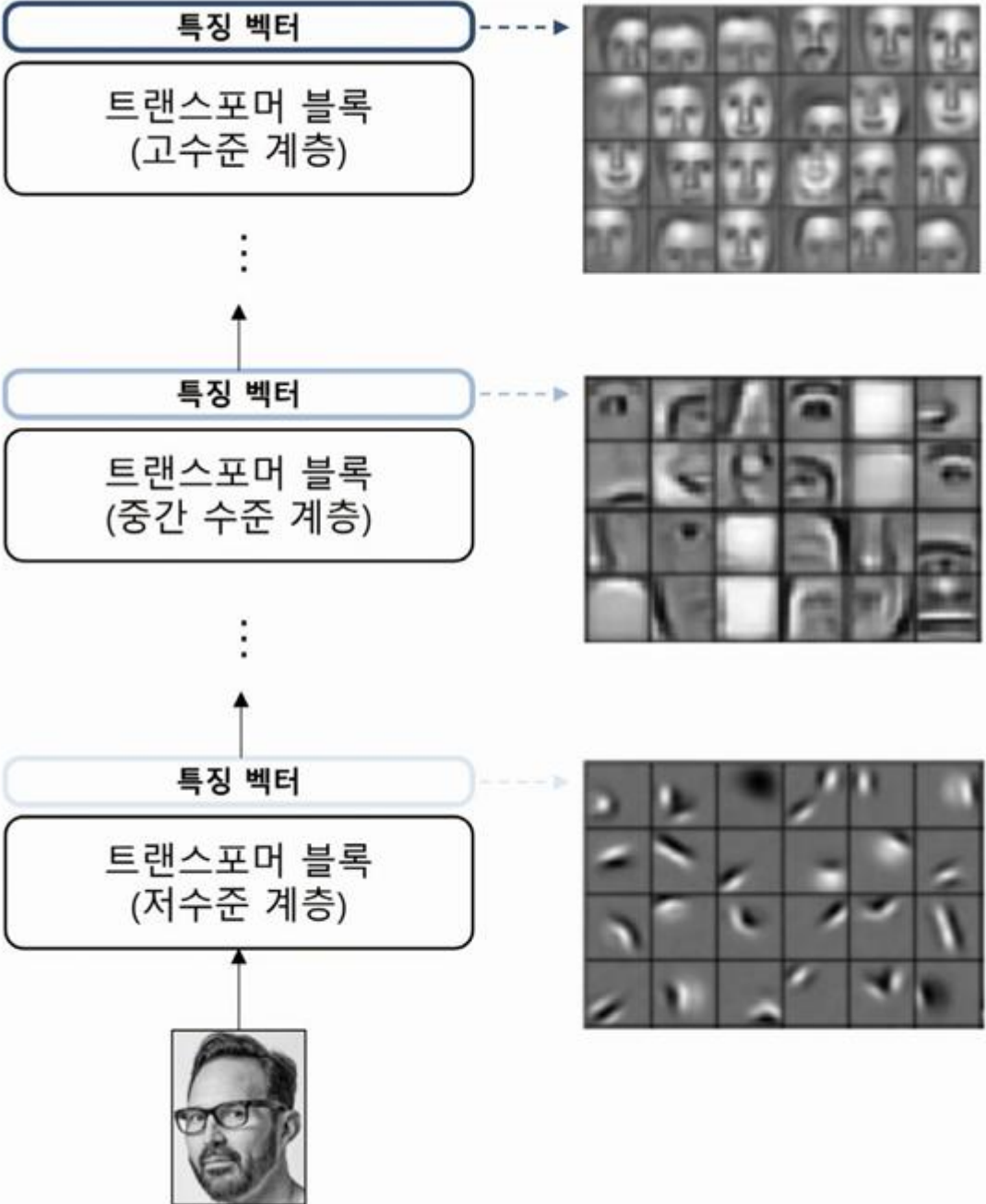
ViT의 문제점: 이미지 정보를 바탕으로 각 패치 간 관계를 학습하지만, 모든 패치에 대해 동일한 크기와 간격을 사용

-> 물체의 크기나 해상도를 계층적으로 학습하기 어려움

CvT의 해결책: 합성곱 신경망의 계층적 구조를 ViT에 적용해 이미지의 지역 특징(합성곱 계층)과 전역 특징(ViT) 모두 활용 -> 비교적 적은 수의 매개변수로 높은 성능 보임

#10.3 CvT

얼굴 이미지가 CvT 모델의 입력일 경우



저수준 -> 선 / 점
중간 수준 -> 눈, 귀
고수준 -> 얼굴 구조

⇒ CvT의 계층형 구조는 물체의 크기나 해상도를 더 잘 표현

그림 10.14 CvT 모델의 계층적 특징 구조 예시

#10.3 CvT

ViT: 이미지를 패치 단위로 분할하고, 이를 트랜스포머 모델 입력으로 사용하는 방식
이때 패치를 겹치지 않게 처리해 모델이 이미지의 전체 정보 학습함

CvT: 이미지 처리 전에 합성곱 연산을 통해 특징 맵 추출 -> 유의미한 패턴 추출하고 일정한 크기의
패치로 나눠 트랜스포머 모델의 입력으로 사용
입력되는 패치들은 겹쳐 학습

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

∴ ViT는 패치 임베딩과 선형 임베딩 사용하지만 CvT 모델은 합성곱 연산을 통해 이미지 학습
CvT는 토큰에 대한 위치 임베딩 적용하지 않아도 모델의 성능 저하 없이 다양한 해상도를 가진
이미지 처리 가능

#10.3 CvT

1. 합성곱 토큰 임베딩

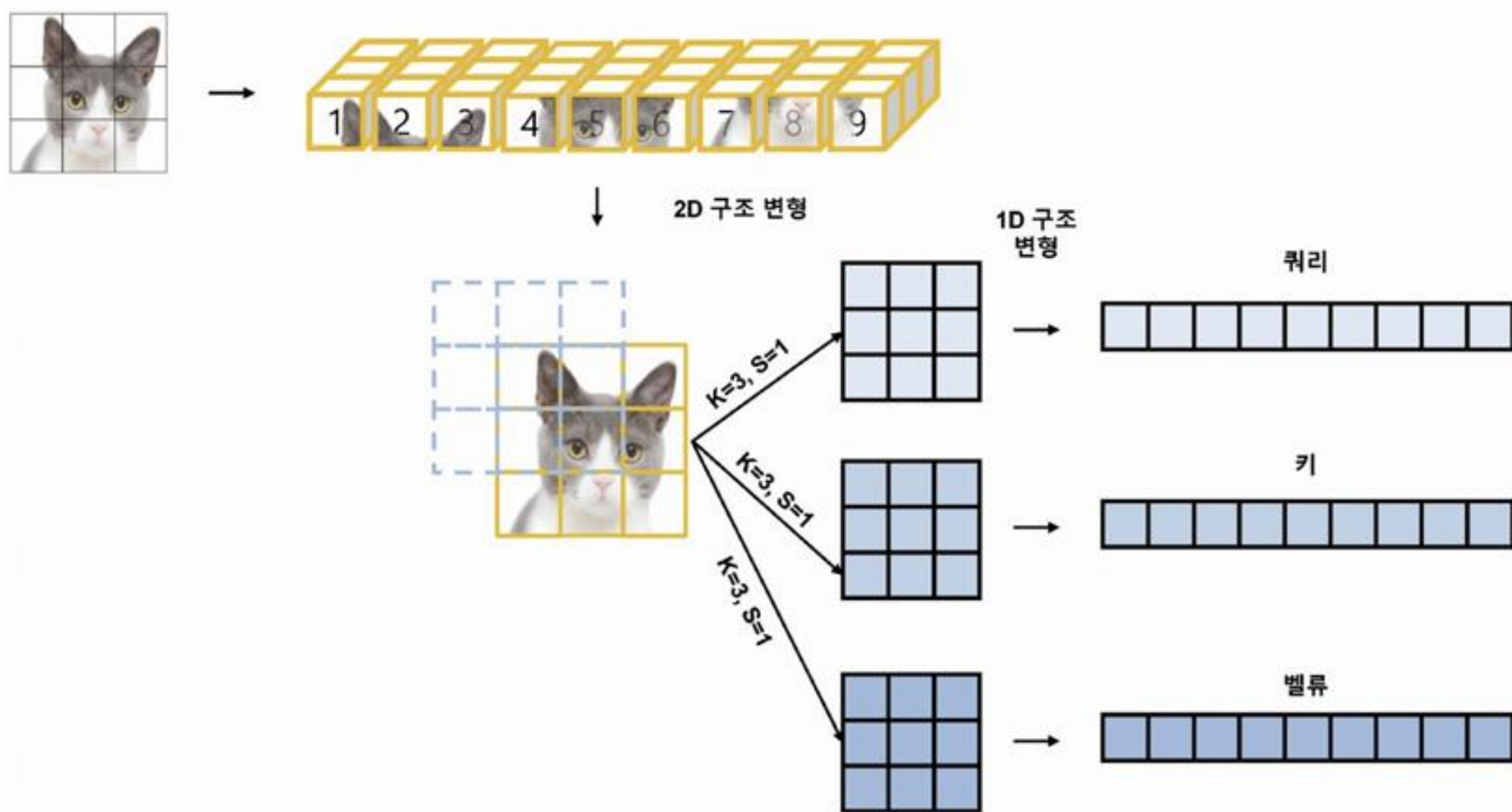
합성곱 토큰 임베딩: 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩
ViT에 계층적인 합성곱 신경망 성질을 추가하기 위해 사용

기존 트랜스포머 모델의 문제점: 임베딩을 사용해 셀프 어텐션을 계산하고 출력을 생성하므로
이미지의 근본적인 2D 모양 구조를 잃게 됨

CvT의 해결책: 이미지의 2D 구조 보존하면서도 셀프 어텐션 계산
2D 합성곱 임베딩을 사용해 이미지 패치를 쿼리, 키, 값 임베딩으로 변환
이렇게 변환된 임베딩을 사용해 셀프 어텐션 계산 & 출력 생성

#10.3 CvT

2D 합성곱 기반의 어텐션 임베딩



3 x 3 커널 & 1 스트라이드

➔ 패치 크기 3 x 3

각각의 패치를 쿼리, 키, 벨류에 대해 합성곱 연산하고 1차원 구조로 재배열해 기존 셀프 어텐션 방법으로 학습 -> 지역 특징 학습 가능, 시퀀스 길이 줄임, 토큰 특징 차원 증가

#10.3 CvT

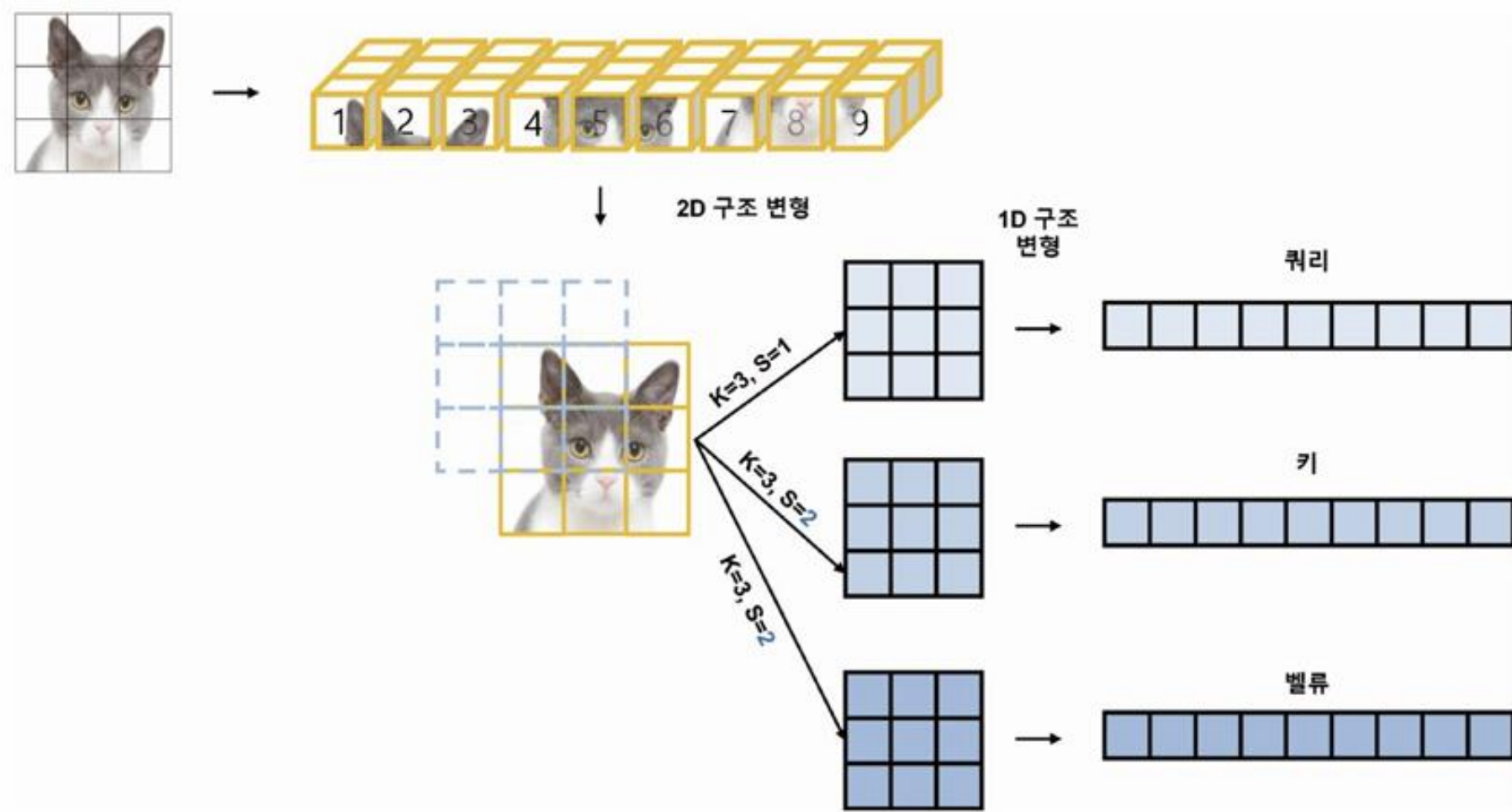
2. 어텐션에 대한 합성곱 임베딩

어텐션에 대한 합성곱 임베딩: 2D로 재구성된 토큰 맵에서 분리 가능한 합성곱 연산을 적용해 성능 저하 및 계산 복잡도를 낮추는 연산

일반적으로 쿼리, 키, 값 임베딩은 차원을 동일하게 설정하지만
CvT는 매개변수 수를 줄이기 위해 쿼리, 키, 값의 차원을 축소시킴

#10.3 CvT

2D 합성곱 기반의 축소된 어텐션 임베딩



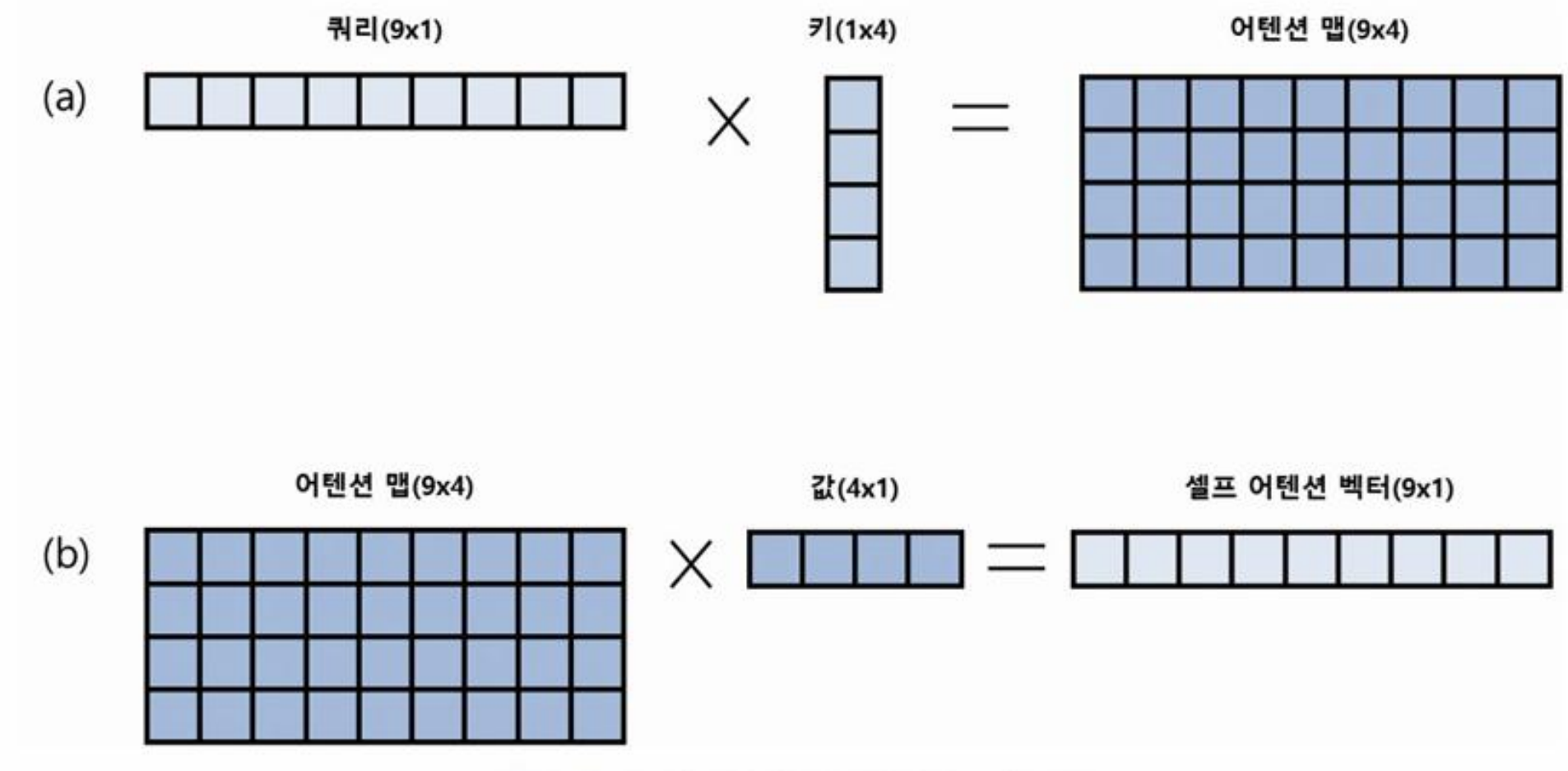
쿼리는 스트라이드 1 -> 9개 이미지 패치

키와 값은 스트라이드 2 -> 4개

이미지 패치

#10.3 CvT

축소된 어텐션 임베딩(열 x 행)



쿼리는 스트라이드 1 -> 9개 이미지 패치
키와 값은 스트라이드 2 -> 4개 이미지 패치

(a)에서는 9개의 쿼리 패치 임베딩과 4개의 키 패치 임베딩에 대한 중요도를 의미하는 어텐션 맵 생성

어텐션 맵이 생성되면 (b)와 같이 값 패치 임베딩으로 쿼리 어텐션 맵에 대한 가중치 반영
이런 벡터의 출력 벡터의 패치 길이 등이 계산해야 하는 이미지 패치 개수 주의점이

#10.3 CvT

CvT 모델은 트랜스포머 계층이 깊어질수록 패치 간의 관계를 학습하는 길이를 축소시키는 대신 벡터의 차원을 증가시켜 모델의 매개변수 수를 대폭 감소시킴

이미지 패치 크기가 줄어든 만큼 임베딩 차원을 늘리는 방법을 택해 네트워크의 안정성을 향상시키고 모델 설계를 단순화함

#10.3 CvT

3. 모델 실습

ViT 모델의 실습 코드를 활용해 CvT 모델을 미세 조정할 것

CvT는 허깅 페이스 라이브러리에서 제공하는 사전 학습된 Microsoft/cvt-21 모델 사용

Microsoft/cvt-21 모델: ImageNet-1k 데이터셋을 사용해 224 x 224 크기로 학습됨

#10.3 CvT

데이터 전처리

```
image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path = "microsoft/cvt-21"
)

transform = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Resize(
            size=(
                image_processor.size["shortest_edge"],
                image_processor.size["shortest_edge"]
            )
        ),
        transforms.Lambda(
            lambda x: torch.cat([x, x, x], 0)
        ),
        transforms.Normalize(
            mean=image_processor.image_mean,
            std=image_processor.image_std
        )
    ]
)
```

Microsoft/cvt-21 모델로 변경

이미지 크기 조절할 때 shortest_edge 키 사용해
전처리

Shortest_edge: 이미지의 너비나 높이 중 더 작은 값

#10.3 CvT

사전 학습된 CvT 모델 불러오기 / 모델 구조 출력

```
from transformers import CvtForImageClassification

model = CvtForImageClassification.from_pretrained(
    pretrained_model_name_or_path="microsoft/cvt-21",
    num_labels=len(train_dataset.classes),
    id2label={idx: label for label, idx in train_dataset.class_to_idx.items()},
    label2id=train_dataset.class_to_idx,
    ignore_mismatched_sizes=True
)

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("  L", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("    L", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("      L", sssub_name)
```

```
cvt
├── encoder
│   ├── stages
│   │   ├── 0
│   │   ├── 1
│   │   └── 2
│   ├── layernorm
│   └── classifier
```

CvT 모델은 세 개의 스테이지로 구성된 cvt, 계층 정규화, 분류기로 구성됨

CvT는 합성곱 계층을 사용해 지역 특징 정보와 전역 정보를 파악하므로 풀링 적용하지 않음

이미지 사이즈는 스테이지별로 4배, 8배, 16배 감소되며 반비례하게 임베딩 차원이 커짐

#10.3 CvT

첫 번째 스테이지 구조 출력

```
stages = model.cvt.encoder.stages
print(stages[0])
```

```
CvtStage( (embedding): CvtEmbeddings( (convolution_embeddings):
CvtConvEmbeddings( (projection): Conv2d(3, 64, kernel_size=(7, 7), stride=(4, 4),
padding=(2, 2)) (normalization): LayerNorm((64,)), eps=1e-05,
elementwise_affine=True) ) (dropout): Dropout(p=0.0, inplace=False) ) (layers):
Sequential( (0): CvtLayer( (attention): CvtAttention( (attention):
CvtSelfAttention( (convolution_projection_query):
CvtSelfAttentionProjection( (convolution_projection):
CvtSelfAttentionConvProjection( (convolution): Conv2d(64, 64, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=64, bias=False) (normalization):
BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True) )
(linear_projection): CvtSelfAttentionLinearProjection() )
(convolution_projection_key):
CvtSelfAttentionProjection( (convolution_projection):
CvtSelfAttentionConvProjection( (convolution): Conv2d(64, 64, kernel_size=(3, 3),
stride=(2, 2), padding=(1, 1), groups=64, bias=False) (normalization):
BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True) )
(linear_projection): CvtSelfAttentionLinearProjection() )
(convolution_projection_value):
CvtSelfAttentionProjection( (convolution_projection):
CvtSelfAttentionConvProjection( (convolution): Conv2d(64, 64, kernel_size=(3, 3),
stride=(2, 2), padding=(1, 1), groups=64, bias=False) (normalization):
BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True) )
(linear_projection): CvtSelfAttentionLinearProjection() ) (projection_query):
Linear(in_features=64, out_features=64, bias=True) (projection_key):
Linear(in_features=64, out_features=64, bias=True) (projection_value):
Linear(in_features=64, out_features=64, bias=True) (dropout): Dropout(p=0.0,
inplace=False) ) (output): CvtSelfOutput( (dense): Linear(in_features=64,
out_features=64, bias=True) (dropout): Dropout(p=0.0, inplace=False) ) )
(intermediate): CvtIntermediate( (dense): Linear(in_features=64,
out_features=256, bias=True) (activation): GELU(approximate='none') ) (output):
CvtOutput( (dense): Linear(in_features=256, out_features=64, bias=True)
(dropout): Dropout(p=0.0, inplace=False) ) (drop_path): Identity()
(layer_norm_before): LayerNorm((64,)), eps=1e-05, elementwise_affine=True)
(layer_norm_after): LayerNorm((64,)), eps=1e-05, elementwise_affine=True) ) ) )
```

각 스테이지는 크게 CvtEmbeddings, CvtLayer로 구성

CvtEmbeddings는 입력 이미지에 대한 2D 합성곱
연산과 계층 정규화로 이루어져 있으며 이미지를 패치
단위로 잘라 임베딩하는 역할

입력 이미지는 첫 번째 스테이지의 CvtEmbeddings에
적용되는데, 7x7 커널 크기, 4 간격, 2 패딩 크기로
합성곱 연산 적용

#10.3 CvT

셀프 어텐션 모듈에 이미지 전달했을 때의 셀프 어텐션 결과값

```
batch = next(iter(train_dataloader))
print("이미지 차원 :", batch["pixel_values"].shape)

patch_emb_output = stages[0].embedding(batch["pixel_values"])
print("패치 임베딩 차원 :", patch_emb_output.shape)

batch_size, num_channels, height, width = patch_emb_output.shape
hidden_state = patch_emb_output.view(batch_size, num_channels, height * width).permute(0, 2, 1)
print("셀프 어텐션 입력 차원 :", hidden_state.shape)

attention_output = stages[0].layers[0].attention.attention(hidden_state, height, width)
print("셀프 어텐션 출력 차원 :", attention_output.shape)
```

```
이미지 차원 : torch.Size([32, 3, 224, 224])
패치 임베딩 차원 : torch.Size([32, 64, 56, 56])
셀프 어텐션 입력 차원 : torch.Size([32, 3136, 64])
셀프 어텐션 출력 차원 : torch.Size([32, 3136, 64])
```

합성곱 계층을 통해 계산된 패치
임베딩은 가로와 세로 크기로
재정렬해 CvTLayer에 입력됨

CvTLayer의 attention.attention 계층을
통해 셀프 어텐션이 수행됨

모듈의 입력 차원과 출력 차원 형태
동일

#10.3 CvT

FashionMNIST 데이터셋 활용해 CvT 모델 미세조정

```
def model_init(classes, class_to_idx):
    model = CvtForImageClassification.from_pretrained(
        pretrained_model_name_or_path="microsoft/cvt-21",
        num_labels=len(classes),
        id2label={idx: label for label, idx in class_to_idx.items()},
        label2id=class_to_idx,
        ignore_mismatched_sizes=True
    )
    return model
```

```
image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path="microsoft/cvt-21"
)

transform = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Resize(
            size=(
                image_processor.size["shortest_edge"],
                image_processor.size["shortest_edge"]
            )
        ),
        transforms.Lambda(
            lambda x: torch.cat([x, x, x], 0)
        ),
        transforms.Normalize(
            mean=image_processor.image_mean,
            std=image_processor.image_std
        )
    ]
)
```

```
args = TrainingArguments(
    output_dir="..../models/CvT-FashionMNIST",
    save_strategy="epoch",
    eval_strategy="epoch",
    learning_rate=1e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.001,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    # logging_dir 제거
    logging_steps=125,
    remove_unused_columns=False,
    seed=7
)
```

훈련자 클래스(Trainer)를 통해 CvT 모델 학습 수행

학습 완료 후 CvT-FashionMNIST 폴더에 체크포인트마다 학습 결과, 성능 지표, 하이퍼파라미터 등이 저장됨

Epoch	Training Loss	Validation Loss	F1
1	0.796623	0.328437	0.893352
2	0.679725	0.263936	0.909995
3	0.663063	0.255343	0.918896

#10.3 CvT

비전 트랜스포머 모델 비교

	ViT	스원 트랜스포머	CvT
모델 크기	327,325 MB	105,227 MB	120,791 MB
F1-점수	0.9231	0.9159	0.9173
초당 처리 수	29,636 eval/s	58,048 eval/s	44,778 eval/s

비전 트랜스포머 계열 모델은 작은 크기의 학습 데이터셋에서도 높은 정확도와 우수한 성능을 보임

일반화 성능이 가장 좋음: CvT

모델 크기 대비 가장 성능 우수: 스원 트랜스포머

F1-점수가 가장 높음: ViT

-> 비전 트랜스포머 모델은 분류 문제 해결에 있어 매우 유용

THANK YOU

